

Quickstart Guide

Get the HR Screening Tool running on a fresh Windows machine from scratch.

Step 1 — Check prerequisites

Open **Windows PowerShell** and move into your Documents folder:

```
cd $HOME\Documents
```

Then run each check below.

uv (Python package manager)

```
uv --version
```

Should print a version (e.g. `uv 0.11.x`). If you get "not recognized", install it:

```
winget install astral-sh.uv --accept-package-agreements --accept-source-agreements
```

Close and re-open PowerShell after installing.

Python 3.12

You do **not** need to install Python from python.org or touch any PATH variables — uv manages Python for you. Install a standalone Python 3.12 build with one command:

```
uv python install 3.12
```

uv stores this Python in its own directory and uses it automatically via `uv run` and `uv sync`. (This step is optional — `uv sync` in Step 4 auto-downloads Python 3.12 if it's missing — but running it now makes the first sync faster.)

Claude Code

```
where.exe claude
```

If you see a path — Claude is installed. If you get "Could not find files", install it with the official PowerShell installer (this is the recommended native install; it adds `claude` to your PATH automatically — no manual environment-variable editing needed):

```
irm https://claude.ai/install.ps1 | iex
```

Close and re-open PowerShell after installing, then confirm with `where.exe claude`. The native install auto-updates in the background, so you stay on the latest version.

Claude Code requires a Pro, Max, Team, or Enterprise account. On first launch, run `cclaude` once and follow the browser prompt to sign in.

winget (package manager)

```
winget --version
```

Should print a version number (e.g. `v1.28.x`). winget ships with Windows 11 by default. If missing, update the **App Installer** package from the Microsoft Store.

Step 2 — Install Visual C++ 2022 Redistributable

PyTorch (required by docling) needs the **2022** runtime. The 2019 version that ships with many machines is not sufficient.

```
winget install Microsoft.VCRedist.2015+.x64 --accept-package-agreements --accept-source-agreements
```

This is safe to run even if already installed — winget will report "already installed" and exit cleanly.

Step 3 — Clone the repository

```
git clone https://github.com/DysrupIT-AI-Lab-Internal-Team/HR-Screening---v2-claude-windows.git
cd HR-Screening---v2-claude-windows
```

Allow PowerShell scripts (such as `run.ps1` and `setup.ps1`) to run for your user. Many Windows machines block scripts by default, so run this once:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

This applies only to your user account and does not require administrator rights.

Step 4 — Install dependencies

Dependencies are declared in `pyproject.toml` and pinned in `uv.lock`. Install them with:

```
uv sync
```

`uv sync` creates a Python 3.12 environment automatically (downloading a standalone Python 3.12 build if the machine doesn't have one — no winget/python.org install needed) and installs the exact locked versions

(`docling`, `colorama`, `python-dotenv`, `pypdf`). PyTorch is pulled in automatically as a transitive dependency of `docling`, so `docling`'s OCR / layout pipeline works out of the box.

You never create or activate a virtual environment by hand and never call `pip` — `uv` manages everything. You run the tool later with `uv run` or `.\run.ps1` (Step 9).

The **first** time you do this, `uv` downloads PyTorch (~120 MB wheel), Transformers, and several ML libraries into its global cache. On any later sync — this project or another — `uv` hardlinks the same cached wheels instead of re-downloading or re-copying them, so it's near-instant and uses no extra disk space.

The tool runs on CPU — no GPU setup is required.

Verify the environment uses the correct Python version:

```
uv run python --version
```

Expected output: Python 3.12.x

Step 5 — Verify the installation

Run these checks one at a time. Each should print OK with no errors.

Check `docling` + torch (PDF support)

```
uv run python -c "import torch; from docling.document_converter import DocumentConverter; print('docling + torch OK')"
```

If this raises `ModuleNotFoundError: No module named 'torch'`, the sync didn't complete. Re-run Step 4 (`uv sync`).

Check Claude CLI is detected

```
uv run python -c "
import shutil, os
from pathlib import Path
hit = shutil.which('claude')
if not hit:
    base = Path(os.environ.get('LOCALAPPDATA', '')) / 'Packages'
    for pkg in base.iterdir():
        if 'claude' in pkg.name.lower():
            for sub in sorted((pkg / 'LocalCache' / 'Roaming' / 'Claude' / 'claude-code').iterdir(), key=lambda x: x.name):
                exe = sub / 'claude.exe'
                if exe.is_file():
                    hit = str(exe)
                    break
print('Claude CLI:', hit or 'NOT FOUND')
"
```

Expected output: Claude CLI: C:\Users\...\claude.exe

If it prints NOT FOUND, re-run the Claude installer from Step 1 (`irm https://claude.ai/install.ps1 | iex`),

re-open PowerShell, and try again.

Step 6 — Pre-download the OCR / layout models (recommended)

On the **first** PDF it processes, docling downloads its ML model weights from the internet — the RapidOCR detection/classification/recognition models (~40 MB from modelscope.cn) plus docling's layout and table-structure models (from Hugging Face). This happens automatically, but it means the *first* resume is slow and **requires an internet connection**.

To get this out of the way now (and confirm processing will work), pre-fetch the models with docling's downloader:

```
uv run docling-tools models download
```

The models are cached under your user profile, so this only needs to run once per machine. If the machine that runs screenings has no internet access, run this step on it while it is still online — processing itself works offline once models are cached.

Step 7 — Set up folders

Create the required input folders if they don't exist:

```
New-Item -ItemType Directory -Force resumes, jd, results
```

- Drop **resume files** (.pdf or .txt) into subfolders under `resumes\`
e.g. `resumes\DevOps Engineer\john_doe.pdf`
- Drop **job description files** (.txt or .md) into `jd\`
e.g. `jd\devops_engineer.txt`

Step 8 — Configure the environment

Copy the example config to a local `.env` file:

```
copy .env.example .env
```

`.env` is git-ignored and holds local configuration. The only setting today is `CLAUDE_MODEL` — leave it blank to use Claude Code's default model, or set a specific model name (e.g. `claude-sonnet-4-6`) to override.

Step 9 — Run the tool

```
.\run.ps1
```

run.ps1 launches the tool via uv run, which syncs the environment from uv.lock and runs inside it — no activation needed. You can also run it directly:

```
uv run hr_screening_tool_v2_win.py
```

You should see the main menu:

```
=====
  HR SCREENING TOOL v2  --  Windows Optimized
  Powered by Claude Code Pro
  PDF extraction: Docling -- OCR + layout-aware Markdown
=====

Main Menu
[1] Screen New Resumes ...
```

Troubleshooting

Symptom	Cause	Fix
docling not installed / PDF support disabled	Dependencies not synced, or running system Python instead of the uv environment	Re-run Step 4 (uv sync); launch via .\run.ps1 or uv run hr_screening_tool_v2_win.py, not python hr_screening_tool_v2_win.py
ModuleNotFoundError: No module named 'torch'	uv sync did not complete	Re-run uv sync from the project root
First resume hangs / connection errors during processing	OCR/layout models downloading on first run	Run Step 6 while online; needs internet on first use
OSError: [WinError 1114] c10.dll	Missing Visual C++ 2022 runtime	Re-run Step 2
Claude Code is not installed or not in PATH	Claude Code not installed, or PATH stale	Run irm https://claude.ai/install.ps1 iex, then re-open PowerShell
uv not recognized	uv not installed or PATH stale	winget install astral-sh.uv, then re-open PowerShell
uv sync fails to find / create the environment	Not in the project folder, or Python 3.12 unavailable	Run uv commands from the project root; uv auto-downloads Python 3.12
First install is slow / downloads a lot	Cold uv cache (first machine ever)	One-time — later syncs hardlink from the cache instantly